Lab assignment-11.2

Name: D.BHARATH KUMAR

Hallticket:2303A51097

Batch-02

<u>Task Description #1 – Stack Implementation</u>

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty

methods.

Sample Input Code:

class Stack:

    pass

Expected Output:

- A functional stack implementation with all required methods and

  docstrings.

Prompt:

#genarate a code to implement all stack operations like push , pop, peek, isEmpty and display using list in python

Code:

assg_11.py > ...

```python
#genarate a code to implement all stack operations like push , pop, peek, isEmpty and display using list in python
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.stack.append(item)
        print(f"Item '{item}' pushed to stack.")

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
        if not self.is_empty():
            item = self.stack.pop()
            print(f"Item '{item}' popped from stack.")
            return item
        else:
            print("Stack is empty. Cannot pop item.")
            return None

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
        if not self.is_empty():
            item = self.stack[-1]
            print(f"Item at top of stack: '{item}'")
            return item
        else:
            print("Stack is empty. Cannot peek item.")
            return None

    def is_empty(self):
        """Return True if the stack is empty, False otherwise."""
        return len(self.stack) == 0

    def display(self):
        """Display all items in the stack."""
        if not self.is_empty():
            print("Stack contents:")
            for item in reversed(self.stack):
                print(item)
        else:
            print("Stack is empty.")
# Example usage
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    stack.display()
    stack.peek()
    stack.pop()
    stack.display()
    print("Is stack empty?", stack.is_empty())
    stack.pop()
    stack.pop()
    print("Is stack empty?", stack.is_empty())
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
Item '1' pushed to stack.
Item '2' pushed to stack.
Item '3' pushed to stack.
Stack contents:
3
2
1
Item at top of stack: '3'
Item '3' popped from stack.
Stack contents:
2
1
Is stack empty? False
Item '2' popped from stack.
Item '1' popped from stack.
Is stack empty? True
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Prompt:

#generate a code to implement queue operations like enqueue, dequeue, peek, isEmpty and display using list in python

Code:

;

```python
#generate a code to implement queue operations like enqueue, dequeue, peek, isEmpty and display using list in python
class Queue:
    | Add to chat ( Alt+X ) | Do not show again
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)
        print(f"Item '{item}' enqueued to queue.")

    def dequeue(self):
        if not self.is_empty():
            item = self.queue.pop(0)
            print(f"Item '{item}' dequeued from queue.")
            return item
        else:
            print("Queue is empty. Cannot dequeue item.")
            return None

    def peek(self):
        if not self.is_empty():
            item = self.queue[0]
            print(f"Item at front of queue: '{item}'")
            return item
        else:
            print("Queue is empty. Cannot peek item.")
            return None

    def is_empty(self):
        return len(self.queue) == 0

    def display(self):
        if not self.is_empty():
            print("Queue contents:")
            for item in self.queue:
                print(item)
        else:
            print("Queue is empty.")
# Example usage
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    queue.display()
    queue.peek()
    queue.dequeue()
    queue.display()
    print("Is queue empty?", queue.is_empty())
    queue.dequeue()
    queue.dequeue()
    print("Is queue empty?", queue.is_empty())
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
Item '1' enqueued to queue.
Item '2' enqueued to queue.
Item '3' enqueued to queue.
Queue contents:
1
2
3
Item at front of queue: '1'
Item '1' dequeued from queue.
Queue contents:
2
3
Is queue empty? False
Item '2' dequeued from queue.
Item '3' dequeued from queue.
Is queue empty? True
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display

methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

Prompt:

#write a ptython code to implemwent to generate a Singly Linked List with insert and display methods

Code:

```python
#write a ptython code to implemwent to generate a Singly Linked List with insert and display methods
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            print(f"Inserted head node with value: {data}")
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
            print(f"Inserted node with value: {data} at the end of the list")

    def display(self):
        if self.head is None:
            print("The linked list is empty.")
        else:
            print("Linked List contents:")
            current = self.head
            while current:
                print(current.data)
                current = current.next
# Example usage
if __name__ == "__main__":
    linked_list = SinglyLinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)
    linked_list.display()
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
Inserted '1' as head of the list.
Inserted '2' at the end of the list.
Inserted '3' at the end of the list.
Singly Linked List contents:
1
2
3
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

<u>Task Description #4 – Binary Search Tree (BST)</u>

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

    pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

Promot:

#write a python code to implement the binary search tree with insert and in-order traversal methods.

Code:

```python
#write a python code to implement the binary search tree with insert and in-order traversal methods.
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = TreeNode(key)
            print(f"Inserted root node with value: {key}")
        else:
            self._insert_recursively(self.root, key)

    def _insert_recursively(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = TreeNode(key)
                print(f"Inserted node with value: {key} to the left of {node.val}")
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = TreeNode(key)
                print(f"Inserted node with value: {key} to the right of {node.val}")
            else:
                self._insert_recursively(node.right, key)

    def in_order_traversal(self):
        print("In-order traversal of the binary search tree:")
        self._in_order_recursively(self.root)

    def _in_order_recursively(self, node):
        if node is not None:
            self._in_order_recursively(node.left)
            print(node.val)
            self._in_order_recursively(node.right)
# Example usage
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    bst.in_order_traversal()
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
Inserted root node with value: 5
Inserted node with value: 3 to the left of 5
Inserted node with value: 7 to the right of 5
Inserted node with value: 2 to the left of 3
Inserted node with value: 4 to the right of 3
Inserted node with value: 6 to the left of 7
Inserted node with value: 8 to the right of 7
In-order traversal of the binary search tree:
2
3
4
5
6
7
8
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

*Task **Description** #5 – Hash Table*

Task: Use AI to implement a hash table with basic insert, search, and

delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

Prompt:

#wirte a python code to implement the hash table to implement a hash table with basic insert, search, and delete methods.

Code:

```python
#wirte a python code to implement the hash table to implement a hash table with basic insert, search, and delete methods.
class HashTable:
    def __init__(self):
        self.table = {}

    def insert(self, key, value):
        self.table[key] = value
        print(f"Inserted key: '{key}' with value: '{value}'")

    def search(self, key):
        if key in self.table:
            print(f"Key: '{key}' found with value: '{self.table[key]}'")
            return self.table[key]
        else:
            print(f"Key: '{key}' not found.")
            return None

    def delete(self, key):
        if key in self.table:
            del self.table[key]
            print(f"Key: '{key}' deleted from hash table.")
            return True
        else:
            print(f"Key: '{key}' not found. Cannot delete.")
            return False
# Example usage
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.search("name")
    hash_table.search("age")
    hash_table.delete("name")
    hash_table.search("name")
    hash_table.search("age")
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
Inserted key: 'name' with value: 'Alice'
Inserted key: 'age' with value: '30'
Key: 'name' found with value: 'Alice'
Key: 'age' found with value: '30'
Key: 'name' deleted from hash table.
Key: 'name' not found.
Key: 'age' found with value: '30'
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

<u>Task Description #6 – Graph Representation</u>

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Prompt:

#write a python code to implement a graph using an adjacency list.

```python
#write a python code to implement a graph using an adjacency list.
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_edge(self, vertex1, vertex2):
        if vertex1 not in self.adjacency_list:
            self.adjacency_list[vertex1] = []
        if vertex2 not in self.adjacency_list:
            self.adjacency_list[vertex2] = []
        self.adjacency_list[vertex1].append(vertex2)
        self.adjacency_list[vertex2].append(vertex1)
        print(f"Edge added between '{vertex1}' and '{vertex2}'")

    def display(self):
        print("Graph adjacency list:")
        for vertex, neighbors in self.adjacency_list.items():
            print(f"{vertex}: {', '.join(neighbors)}")
# Example usage
if __name__ == "__main__":
    graph = Graph()
    graph.add_edge("A", "B")
    graph.add_edge("A", "C")
    graph.add_edge("B", "D")
    graph.add_edge("C", "D")
    graph.display()
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
Edge added between 'A' and 'B'
Edge added between 'A' and 'C'
Edge added between 'B' and 'D'
Edge added between 'C' and 'D'
Graph adjacency list:
A: B, C
B: A, D
C: A, D
D: B, C
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq

module.

Sample Input Code:

```
class PriorityQueue:
    pass
```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority),

  and display methods.

Prompt:

#write a python code to implement a priority queue using Python's heapq module.

Code:

```python
#write a python code to implement a priority queue using Python’s heapq module.
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def push(self, item, priority):
        heapq.heappush(self.elements, (priority, item))
        print(f"Item '{item}' with priority '{priority}' added to the priority queue.")

    def pop(self):
        if not self.is_empty():
            priority, item = heapq.heappop(self.elements)
            print(f"Item '{item}' with priority '{priority}' popped from the priority queue.")
            return item
        else:
            print("Priority queue is empty. Cannot pop item.")
            return None

    def peek(self):
        if not self.is_empty():
            priority, item = self.elements[0]
            print(f"Item at top of priority queue: '{item}' with priority '{priority}'")
            return item
        else:
            print("Priority queue is empty. Cannot peek item.")
            return None

    def is_empty(self):
        return len(self.elements) == 0

    def display(self):
        if not self.is_empty():
            print("Priority Queue contents:")
            for priority, item in sorted(self.elements):
                print(f"Item: '{item}', Priority: '{priority}'")
        else:
            print("Priority queue is empty.")
# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.push("Task 1", 2)
    pq.push("Task 2", 1)
    pq.push("Task 3", 3)
    pq.display()
    pq.peek()
    pq.pop()
    pq.display()
    print("Is priority queue empty?", pq.is_empty())
    pq.pop()
    pq.pop()
    print("Is priority queue empty?", pq.is_empty())
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
Item 'Task 1' with priority '2' added to the priority queue.
Item 'Task 2' with priority '1' added to the priority queue.
Item 'Task 3' with priority '3' added to the priority queue.
Priority Queue contents:
Item: 'Task 2', Priority: '1'
Item: 'Task 1', Priority: '2'
Item: 'Task 3', Priority: '3'
Item at top of priority queue: 'Task 2' with priority '1'
Item 'Task 2' with priority '1' popped from the priority queue.
Priority Queue contents:
Item: 'Task 1', Priority: '2'
Item: 'Task 3', Priority: '3'
Is priority queue empty? False
Item 'Task 1' with priority '2' popped from the priority queue.
Item 'Task 3' with priority '3' popped from the priority queue.
Is priority queue empty? True
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using

collections.deque.

Sample Input Code:

```
class DequeDS:
    pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

Prompt:

#write a python code to implement a double-ended queue using collections.deque.

Code:

```python
#write a python code to implement a double-ended queue using collections.deque.
from collections import deque
class DoubleEndedQueue:
    def __init__(self):
        self.deque = deque()

    def add_front(self, item):
        self.deque.appendleft(item)
        print(f"Item '{item}' added to the front of the deque.")

    def add_rear(self, item):
        self.deque.append(item)
        print(f"Item '{item}' added to the rear of the deque.")

    def remove_front(self):
        if not self.is_empty():
            item = self.deque.popleft()
            print(f"Item '{item}' removed from the front of the deque.")
            return item
        else:
            print("Deque is empty. Cannot remove item from front.")
            return None

    def remove_rear(self):
        if not self.is_empty():
            item = self.deque.pop()
            print(f"Item '{item}' removed from the rear of the deque.")
            return item
        else:
            print("Deque is empty. Cannot remove item from rear.")
            return None

    def peek_front(self):
        if not self.is_empty():
            item = self.deque[0]
            print(f"Item at front of deque: '{item}'")
            return item
        else:
            print("Deque is empty. Cannot peek front item.")
            return None

    def peek_rear(self):
        if not self.is_empty():
            item = self.deque[-1]
            print(f"Item at rear of deque: '{item}'")
            return item
        else:
            print("Deque is empty. Cannot peek rear item.")
            return None

    def is_empty(self):
        return len(self.deque) == 0

    def display(self):
        if not self.is_empty():
            print("Deque contents:")
            for item in self.deque:
                print(item)
        else:
            print("Deque is empty.")
# Example usage
if __name__ == "__main__":
    deq = DoubleEndedQueue()
    deq.add_rear(1)
    deq.add_rear(2)
    deq.add_front(0)
    deq.display()
    deq.peek_front()
    deq.peek_rear()
    deq.remove_front()
    deq.remove_rear()
    deq.display()
    print("Is deque empty?", deq.is_empty())
    deq.remove_front()
    print("Is deque empty?", deq.is_empty())
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
Item '1' added to the rear of the deque.
Item '2' added to the rear of the deque.
Item '0' added to the front of the deque.
Deque contents:
0
1
2
Item at front of deque: '0'
Item at rear of deque: '2'
Item '0' removed from the front of the deque.
Item '2' removed from the rear of the deque.
Deque contents:
1
Is deque empty? False
Item '1' removed from the front of the deque.
Is deque empty? True
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

<u>Task Description #9 Real-Time Application Challenge – Choose the</u>

Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System

that handles:

1. Student Attendance Tracking – Daily log of students

entering/exiting the campus.

2. Event Registration System – Manage participants in events with

quick search and removal.

3. Library Book Borrowing – Keep track of available books and their

due dates.

4. Bus Scheduling System – Maintain bus routes and stop

connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

Task Description #10: Smart Hospital Management System – Data

Code:

```python
"""

# Campus Resource Management System
# Mapping Features to Data Structures with Justifications

"""
FEATURE MAPPING TABLE:
====================
1. Student Attendance Tracking → Linked List
    Justification: Maintains chronological order of entry/exit logs. Easy to insert at end and traverse
    sequentially for daily reports without needing random access.

2. Event Registration System → Hash Table
    Justification: Provides O(1) average lookup, insertion, and deletion of participants by ID. Enables
    quick search and removal operations for managing registrations efficiently.

3. Library Book Borrowing → BST
    Justification: Books can be organized by ISBN or title for efficient searching. In-order traversal
    provides sorted book catalog, and BST allows quick insertion/deletion of borrowed/returned books.

4. Bus Scheduling System → Graph
    Justification: Represents bus routes as vertices and connections between stops as edges. Models
    complex route networks and enables pathfinding between different campus stops.

5. Cafeteria Order Queue → Queue (FIFO)
    Justification: Students must be served in arrival order. Queue's FIFO nature ensures fairness and
    efficient management of student orders without priority bias.
"""

# IMPLEMENTATION: Library Book Borrowing System using Binary Search Tree

class BookNode:
    """Node class for BST storing book information"""
    def __init__(self, isbn, title, author):
        self.isbn = isbn
        self.title = title
        self.author = author
        self.left = None
        self.right = None
        self.is_available = True


class LibraryBookSystem:
    """Binary Search Tree implementation for managing library books"""
    def __init__(self):
        self.root = None

    def insert_book(self, isbn, title, author):
        """Insert a new book into the library system"""
        if self.root is None:
            self.root = BookNode(isbn, title, author)
            print(f"📚 Added Book: '{title}' (ISBN: {isbn})")
        else:
            self._insert_recursively(self.root, isbn, title, author)

    def _insert_recursively(self, node, isbn, title, author):
        """Helper method to insert book recursively"""
        if isbn < node.isbn:
            if node.left is None:
                node.left = BookNode(isbn, title, author)
                print(f"📚 Added Book: '{title}' (ISBN: {isbn})")
            else:
                self._insert_recursively(node.left, isbn, title, author)
        else:
            if node.right is None:
                node.right = BookNode(isbn, title, author)
                print(f"📚 Added Book: '{title}' (ISBN: {isbn})")
            else:
                self._insert_recursively(node.right, isbn, title, author)

    def search_book(self, isbn):
        """Search for a book by ISBN"""
        result = self._search_recursively(self.root, isbn)
        if result:
            status = "✅ Available" if result.is_available else "❌ Not Available"
            print(f"Found: '{result.title}' by {result.author} - {status}")
            return result
        else:
            print(f"Book with ISBN {isbn} not found.")
            return None

    def _search_recursively(self, node, isbn):
        """Helper method to search recursively"""
        if node is None:
            return None
        if isbn == node.isbn:
            return node
        elif isbn < node.isbn:
            return self._search_recursively(node.left, isbn)
        else:
            return self._search_recursively(node.right, isbn)

    def borrow_book(self, isbn):
        """Mark a book as borrowed"""
        book = self.search_book(isbn)
        if book and book.is_available:
            book.is_available = False
            print(f"✔ Book borrowed: '{book.title}'")
        elif book:
            print(f"❌ Book '{book.title}' is already borrowed.")

```

```python
    def return_book(self, isbn):
        """Mark a book as returned"""
        book = self.search_book(isbn)
        if book and not book.is_available:
            book.is_available = True
            print(f"✅ Book returned: '{book.title}'")
        elif book:
            print(f"⚠️ Book '{book.title}' was not borrowed.")

    def display_all_books(self):
        """Display all books in sorted order (in-order traversal)"""
        if self.root is None:
            print("📭 Library is empty.")
        else:
            print("\n📚 Library Catalog (Sorted by ISBN):")
            print("-" * 60)
            self._in_order_traversal(self.root)
            print("-" * 60 + "\n")

    def _in_order_traversal(self, node):
        """In-order traversal to display books in sorted order"""
        if node is not None:
            self._in_order_traversal(node.left)
            status = "Available" if node.is_available else "Borrowed"
            print(f"ISBN: {node.isbn} | {node.title} | {node.author} | {status}")
            self._in_order_traversal(node.right)


# Example Usage
if __name__ == "__main__":
    print("=" * 60)
    print("🏫 CAMPUS LIBRARY BOOK MANAGEMENT SYSTEM")
    print("=" * 60 + "\n")

    library = LibraryBookSystem()

    # Add books
    library.insert_book(101, "Python Programming", "Guido van Rossum")
    library.insert_book(105, "Data Structures", "Mark Allen Weiss")
    library.insert_book(98, "Algorithms", "Thomas Cormen")
    library.insert_book(110, "Web Development", "Kyle Simpson")
    library.insert_book(95, "Database Design", "C.J. Date")

    # Display all books
    library.display_all_books()

    # Borrow books
    print("📤 Borrowing Operations:")
    library.borrow_book(101)
    library.borrow_book(105)

    # Try to borrow already borrowed book
    library.borrow_book(101)

    # Display updated catalog
    library.display_all_books()

    # Search for a book
    print("🔍 Search Operations:")
    library.search_book(98)
    library.search_book(999)

    # Return books
    print("\n📥 Returning Operations:")
    library.return_book(101)
    library.return_book(105)

    # Final catalog
    library.display_all_books()
```

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
============================================================
🏫 CAMPUS LIBRARY BOOK MANAGEMENT SYSTEM
============================================================

📚 Added Book: 'Python Programming' (ISBN: 101)
📚 Added Book: 'Data Structures' (ISBN: 105)
📚 Added Book: 'Algorithms' (ISBN: 98)
📚 Added Book: 'Web Development' (ISBN: 110)
📚 Added Book: 'Database Design' (ISBN: 95)

📖 Library Catalog (Sorted by ISBN):
------------------------------------------------------------
ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available
------------------------------------------------------------

📤 Borrowing Operations:
Found: 'Python Programming' by Guido van Rossum - ✅ Available
✔Book borrowed: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - ✅ Available
✔Book borrowed: 'Data Structures'
Found: 'Python Programming' by Guido van Rossum - ❌ Not Available
❌ Book 'Python Programming' is already borrowed.

📖 Library Catalog (Sorted by ISBN):
------------------------------------------------------------
ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Borrowed
ISBN: 105 | Data Structures | Mark Allen Weiss | Borrowed
ISBN: 110 | Web Development | Kyle Simpson | Available
------------------------------------------------------------

🔍 Search Operations:
Found: 'Algorithms' by Thomas Cormen - ✅ Available
```

```
📤 Borrowing Operations:
Found: 'Python Programming' by Guido van Rossum - ✅ Available
✔Book borrowed: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - ✅ Available
✔Book borrowed: 'Data Structures'
Found: 'Python Programming' by Guido van Rossum - ❌ Not Available
❌ Book 'Python Programming' is already borrowed.

📖 Library Catalog (Sorted by ISBN):
------------------------------------------------------------
ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Borrowed
ISBN: 105 | Data Structures | Mark Allen Weiss | Borrowed
ISBN: 110 | Web Development | Kyle Simpson | Available
------------------------------------------------------------

🔍 Search Operations:
Found: 'Algorithms' by Thomas Cormen - ✅ Available
Book with ISBN 999 not found.

📥 Returning Operations:
Found: 'Python Programming' by Guido van Rossum - ❌ Not Available
✔Book returned: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - ❌ Not Available
✔Book returned: 'Data Structures'

📖 Library Catalog (Sorted by ISBN):
------------------------------------------------------------
ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available
------------------------------------------------------------

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

<u>Task Description #10: Smart Hospital Management System – Data</u>

Structure Selection

A hospital wants to develop a Smart Hospital Management System that

handles:

1. Patient Check-In System – Patients are registered and treated in

order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using

ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python
"""
FEATURE MAPPING TABLE:
====================
1. Patient Check-In System → Queue
    Justification: Patients are treated in FIFO order based on arrival time. Queue ensures fair
    service and maintains chronological order for non-emergency patients.

2. Emergency Case Handling → Priority Queue
    Justification: Critical patients must be prioritized regardless of arrival time. Priority Queue
    allows sorting by severity level, ensuring urgent cases are treated first.

3. Medical Records Storage → Hash Table
    Justification: Provides O(1) average lookup time for patient records using patient ID as key.
    Enables fast retrieval and updating of medical information.

4. Doctor Appointment Scheduling → Binary Search Tree (BST)
    Justification: Appointments can be organized by time slots for efficient searching and scheduling.
    In-order traversal provides sorted appointment schedule.

5. Hospital Room Navigation → Graph
    Justification: Represents hospital layout with wards and rooms as vertices and connections as edges.
    Enables pathfinding and navigation between different hospital sections.
"""

# IMPLEMENTATION: Emergency Case Handling using Priority Queue
import heapq

class Patient:
    def __init__(self, patient_id, name, severity, arrival_time):
        self.patient_id = patient_id
        self.name = name
        self.severity = severity  # 1=Critical, 2=High, 3=Medium, 4=Low
        self.arrival_time = arrival_time
        self.status = "Waiting"

    def __lt__(self, other):
        return self.severity < other.severity

    def __repr__(self):
        return f"Patient({self.patient_id}, {self.name}, Severity: {self.severity})"


class EmergencyManagementSystem:
    """Manages emergency patient queue using priority queue."""

    def __init__(self):
        self.emergency_queue = []
        self.treated_patients = []

    def add_patient(self, patient_id, name, severity, arrival_time):
        """Add a patient to the emergency queue."""
        patient = Patient(patient_id, name, severity, arrival_time)
        heapq.heappush(self.emergency_queue, patient)
        severity_map = {1: "🔴 CRITICAL", 2: "🟠 HIGH", 3: "🟡 MEDIUM", 4: "🟢 LOW"}
        print(f"✅ Patient Added: {name} (ID: {patient_id}) - {severity_map[severity]}")

    def treat_patient(self):
        """Treat the next patient with highest priority."""
        if not self.is_empty():
            patient = heapq.heappop(self.emergency_queue)
            patient.status = "Treated"
            self.treated_patients.append(patient)
            severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
            print(f"🏥 Now Treating: {patient.name} (Severity: {severity_map[patient.severity]})")
            return patient
        else:
            print("❌ No patients in queue.")
            return None
```

```python
class EmergencyManagementSystem:
    def peek_next_patient(self):
        """View next patient to be treated."""
        if not self.is_empty():
            patient = self.emergency_queue[0]
            severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
            print(f"👀 Next Patient: {patient.name} (Severity: {severity_map[patient.severity]})")
            return patient
        else:
            print("❌ No patients in queue.")
            return None

    def is_empty(self):
        """Check if queue is empty."""
        return len(self.emergency_queue) == 0

    def display_queue(self):
        """Display all patients waiting in queue."""
        if self.is_empty():
            print("📭 No patients waiting.\n")
        else:
            print("\n🚨 EMERGENCY QUEUE (Sorted by Priority):")
            print("-" * 70)
            severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
            for patient in sorted(self.emergency_queue):
                print(f"ID: {patient.patient_id} | Name: {patient.name} | Severity: {severity_map[patient.severity]} | Status: {patient.status}")
            print("-" * 70 + "\n")

    def display_treated_patients(self):
        """Display all treated patients."""
        if not self.treated_patients:
            print("📭 No treated patients yet.\n")
        else:
            print("\n✅ TREATED PATIENTS:")
            print("-" * 70)
            severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
            for patient in self.treated_patients:
                print(f"ID: {patient.patient_id} | Name: {patient.name} | Severity: {severity_map[patient.severity]}")
            print("-" * 70 + "\n")


# Example Usage
if __name__ == "__main__":
    print("=" * 70)
    print("🏥 SMART HOSPITAL EMERGENCY MANAGEMENT SYSTEM")
    print("=" * 70 + "\n")

    hospital = EmergencyManagementSystem()

    # Add patients with different severity levels
    print("📋 Adding Patients to Emergency Queue:\n")
    hospital.add_patient(1, "John Doe", 4, "09:00")
    hospital.add_patient(2, "Sarah Smith", 1, "09:05")
    hospital.add_patient(3, "Mike Johnson", 3, "09:10")
    hospital.add_patient(4, "Emily Brown", 2, "09:15")
    hospital.add_patient(5, "David Wilson", 1, "09:20")

    # Display queue
    hospital.display_queue()

    # Peek next patient
    print("🔍 Checking Next Patient:\n")
    hospital.peek_next_patient()

    # Treat patients
    print("\n🏥 TREATMENT PROCESS:\n")
    hospital.treat_patient()
    hospital.treat_patient()
    hospital.treat_patient()

    # Display remaining queue
    hospital.display_queue()

    # Display treated patients
    hospital.display_treated_patients()

    # Treat remaining patients
    print("Treating remaining patients:\n")
```

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
======================================================================
🏥 SMART HOSPITAL EMERGENCY MANAGEMENT SYSTEM
======================================================================

📋 Adding Patients to Emergency Queue:

✅ Patient Added: John Doe (ID: 1) - 🟢 LOW
✅ Patient Added: Sarah Smith (ID: 2) - 🔴 CRITICAL
✅ Patient Added: Mike Johnson (ID: 3) - 🟡 MEDIUM
✅ Patient Added: Emily Brown (ID: 4) - 🟠 HIGH
✅ Patient Added: David Wilson (ID: 5) - 🔴 CRITICAL

🚑 EMERGENCY QUEUE (Sorted by Priority):
----------------------------------------------------------------------
ID: 2 | Name: Sarah Smith | Severity: CRITICAL | Status: Waiting
ID: 5 | Name: David Wilson | Severity: CRITICAL | Status: Waiting
ID: 4 | Name: Emily Brown | Severity: HIGH | Status: Waiting
ID: 3 | Name: Mike Johnson | Severity: MEDIUM | Status: Waiting
ID: 1 | Name: John Doe | Severity: LOW | Status: Waiting
----------------------------------------------------------------------

🔍 Checking Next Patient:

👀 Next Patient: Sarah Smith (Severity: CRITICAL)

🏥 TREATMENT PROCESS:

🏥 Now Treating: Sarah Smith (Severity: CRITICAL)
🏥 Now Treating: David Wilson (Severity: CRITICAL)
🏥 Now Treating: Emily Brown (Severity: HIGH)

🚑 EMERGENCY QUEUE (Sorted by Priority):
----------------------------------------------------------------------
ID: 3 | Name: Mike Johnson | Severity: MEDIUM | Status: Waiting
ID: 1 | Name: John Doe | Severity: LOW | Status: Waiting
----------------------------------------------------------------------
```

```
✅ TREATED PATIENTS:
----------------------------------------------------------------------
ID: 2 | Name: Sarah Smith | Severity: CRITICAL
ID: 5 | Name: David Wilson | Severity: CRITICAL
ID: 4 | Name: Emily Brown | Severity: HIGH
----------------------------------------------------------------------

Treating remaining patients:

🏥 Now Treating: Mike Johnson (Severity: MEDIUM)
🏥 Now Treating: John Doe (Severity: LOW)

✅ TREATED PATIENTS:
----------------------------------------------------------------------
ID: 2 | Name: Sarah Smith | Severity: CRITICAL
ID: 5 | Name: David Wilson | Severity: CRITICAL
ID: 4 | Name: Emily Brown | Severity: HIGH
ID: 3 | Name: Mike Johnson | Severity: MEDIUM
ID: 1 | Name: John Doe | Severity: LOW
----------------------------------------------------------------------

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

<u>Task Description #11: Smart City Traffic Control System</u>

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
"""Smart City Traffic Control System.

This module:
1) maps each required feature to the most suitable data structure, and
2) implements Emergency Vehicle Priority Handling using a Priority Queue.

Priority scale used in implementation:
1 = Critical, 2 = High, 3 = Medium, 4 = Low.
"""

from typing import List, Tuple, Optional
from dataclasses import dataclass, field
from itertools import count
import heapq


# Feature -> Data Structure -> Justification (2 sentences each)
FEATURE_MAPPING: List[Tuple[str, str, str]] = [
    (
        "Traffic Signal Queue",
        "Queue",
        "Vehicles at a signal are processed in the same order they arrive. "
        "A queue enforces FIFO behavior and keeps flow fair."
    ),
    (
        "Emergency Vehicle Priority Handling",
        "Priority Queue",
        "Ambulances and fire trucks must move before normal vehicles. "
        "A priority queue always serves the highest-priority request first."
    ),
    (
        "Vehicle Registration Lookup",
        "Hash Table",
        "Vehicle details are usually searched by registration number. "
        "A hash table gives average O(1) lookup, insert, and delete."
    ),
    (
        "Road Network Mapping",
        "Graph",
        "Intersections and roads are naturally represented as nodes and edges. "
        "A graph supports connectivity checks and route finding."
    ),
    (
        "Parking Slot Availability",
        "Hash Table",
        "Each slot ID can map directly to occupied/available status. "
        "A hash table allows quick status checks and updates."
    ),
]


def print_feature_mapping_table() -> None:
    """Print the feature mapping table."""
    print("\nSMART CITY TRAFFIC CONTROL - FEATURE MAPPING")
    print("=" * 120)
    print(f"{'Feature':<36} {'Data Structure':<18} Justification")
    print("-" * 120)
    for feature, structure, reason in FEATURE_MAPPING:
        print(f"{feature:<36} {structure:<18} {reason}")
    print("=" * 120)
```

```python
@dataclass(order=True)
class EmergencyVehicle:
    """Store one emergency vehicle request for priority dispatch."""
    priority: int
    sequence: int
    vehicle_id: str = field(compare=False)
    vehicle_type: str = field(compare=False)
    location: str = field(compare=False)


class EmergencyVehiclePrioritySystem:
    """Priority-based dispatcher for emergency vehicles."""

    def __init__(self) -> None:
        """Initialize an empty emergency priority queue."""
        self._queue: List[EmergencyVehicle] = []
        self._counter = count()

    @staticmethod
    def _label(priority: int) -> str:
        """Convert numeric priority to text."""
        return {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}.get(priority, "UNKNOWN")

    def add_vehicle(self, vehicle_id: str, vehicle_type: str, location: str, priority: int) -> None:
        """Add an emergency vehicle request."""
        req = EmergencyVehicle(
            priority=priority,
            sequence=next(self._counter),
            vehicle_id=vehicle_id,
            vehicle_type=vehicle_type,
            location=location,
        )
        heapq.heappush(self._queue, req)
        print(f"Added: {vehicle_type} ({vehicle_id}) at {location} [{self._label(priority)}]")

    def dispatch_next(self) -> Optional[EmergencyVehicle]:
        """Dispatch and return the highest-priority vehicle."""
        if not self._queue:
            print("No emergency vehicles waiting.")
            return None
        req = heapq.heappop(self._queue)
        print(f"Dispatching: {req.vehicle_type} ({req.vehicle_id}) from {req.location} [{self._label(req.priority)}]")
        return req

    def peek_next(self) -> Optional[EmergencyVehicle]:
        """Show the next vehicle to dispatch without removing it."""
        if not self._queue:
            print("No emergency vehicles waiting.")
            return None
        req = self._queue[0]
        print(f"Next: {req.vehicle_type} ({req.vehicle_id}) at {req.location} [{self._label(req.priority)}]")
        return req

    def is_empty(self) -> bool:
        """Return True if no vehicles are waiting."""
        return len(self._queue) == 0

    def display_queue(self) -> None:
        """Display waiting vehicles in dispatch order."""
        if not self._queue:
            print("Queue is empty.")
            return
```

```python

        print("\nEmergency Queue (highest priority first):")
        print("-" * 80)
        for req in sorted(self._queue):
            print(
                f"ID: {req.vehicle_id:<8} Type: {req.vehicle_type:<12} "
                f"Location: {req.location:<15} Priority: {self._label(req.priority)}"
            )
        print("-" * 80)


if __name__ == "__main__":
    # Print required mapping table
    print_feature_mapping_table()

    # Implemented feature demo: Emergency Vehicle Priority Handling
    system = EmergencyVehiclePrioritySystem()

    print("\nAdding emergency vehicles:")
    system.add_vehicle("AMB-101", "Ambulance", "Junction A", 1)
    system.add_vehicle("FIRE-007", "Fire Truck", "Junction C", 2)
    system.add_vehicle("POL-055", "Police Van", "Junction B", 3)
    system.add_vehicle("AMB-212", "Ambulance", "Junction D", 1)

    system.display_queue()
    system.peek_next()

    print("\nDispatch process:")
    while not system.is_empty():
        system.dispatch_next()
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py

SMART CITY TRAFFIC CONTROL - FEATURE MAPPING
==========================================================================================================
Feature                            Data Structure     Justification
----------------------------------------------------------------------------------------------------------
Traffic Signal Queue               Queue              Vehicles at a signal are processed in the same order they arrive. A queue enforces FIFO behavior and keep
s flow fair.
Emergency Vehicle Priority Handling  Priority Queue   Ambulances and fire trucks must move before normal vehicles. A priority queue always serves the highest-p
riority request first.
Vehicle Registration Lookup        Hash Table         Vehicle details are usually searched by registration number. A hash table gives average O(1) lookup, inse
rt, and delete.
Road Network Mapping               Graph              Intersections and roads are naturally represented as nodes and edges. A graph supports connectivity check
s and route finding.
Parking Slot Availability          Hash Table         Each slot ID can map directly to occupied/available status. A hash table allows quick status checks and u
pdates.
==========================================================================================================

Adding emergency vehicles:
Added: Ambulance (AMB-101) at Junction A [CRITICAL]
Added: Fire Truck (FIRE-007) at Junction C [HIGH]
Added: Police Van (POL-055) at Junction B [MEDIUM]
Added: Ambulance (AMB-212) at Junction D [CRITICAL]

Emergency Queue (highest priority first):
--------------------------------------------------------------------------------
ID: AMB-101  Type: Ambulance    Location: Junction A      Priority: CRITICAL
ID: AMB-212  Type: Ambulance    Location: Junction D      Priority: CRITICAL
ID: FIRE-007 Type: Fire Truck   Location: Junction C      Priority: HIGH
ID: POL-055  Type: Police Van   Location: Junction B      Priority: MEDIUM
--------------------------------------------------------------------------------
Next: Ambulance (AMB-101) at Junction A [CRITICAL]

Dispatch process:
Dispatching: Ambulance (AMB-101) from Junction A [CRITICAL]
Dispatching: Ambulance (AMB-212) from Junction D [CRITICAL]
Dispatching: Fire Truck (FIRE-007) from Junction C [HIGH]
Dispatching: Police Van (POL-055) from Junction B [MEDIUM]
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

Task Description #12: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - o Stack
  - o Queue
  - o Priority Queue
  - o Linked List
  - o Binary Search Tree (BST)
  - o Graph
  - o Hash Table
  - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python

"""
        Smart E-Commerce Platform — Data Structure Challenge
        An e-commerce company wants to build a Smart Online Shopping System
        with:
        1. Shopping Cart Management — Add and remove products dynamically.
        2. Order Processing System — Orders processed in the order they are placed.
        3. Top-Selling Products Tracker — Products ranked by sales count.
        4. Product Search Engine — Fast lookup of products using product ID.
        5. Delivery Route Planning — Connect warehouses and delivery locations.
        """

"""
        FEATURE MAPPING TABLE:
        ====================
        1. Shopping Cart Management → Deque
            Justification: Allows adding/removing products from both ends efficiently.
            Supports undo (pop from end) and redo operations commonly needed in shopping carts.

        2. Order Processing System → Queue
            Justification: Orders must be processed in FIFO order based on arrival time.
            Queue ensures fair processing and maintains chronological order for order fulfillment.

        3. Top-Selling Products Tracker → Priority Queue
            Justification: Products are ranked by sales count with highest-selling items prioritized.
            Priority queue automatically maintains sorted order by sales frequency for quick access to top products.

        4. Product Search Engine → Hash Table
            Justification: Provides O(1) average lookup time for products using product ID as key.
            Enables fast retrieval of product details like price, stock, and specifications.

        5. Delivery Route Planning → Graph
            Justification: Represents warehouses and delivery locations as vertices with roads as edges.
            Graph enables pathfinding algorithms to optimize delivery routes and reduce shipping time.
        """

# IMPLEMENTATION: Shopping Cart Management using Deque


from collections import deque


class Product:
    def __init__(self, product_id, name, price, quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"Product({self.product_id}, {self.name}, ${self.price}, Qty: {self.quantity})"


class ShoppingCart:
    """Shopping Cart implementation using Deque for efficient add/remove operations."""

    def __init__(self):
        self.cart = deque()

    def add_product(self, product):
        """Add a product to the end of the cart."""
        self.cart.append(product)
        print(f"✅ Added to cart: {product.name} - ${product.price}")

```

```python

    def add_product_front(self, product):
        """Add a product to the front of the cart (priority addition)."""
        self.cart.appendleft(product)
        print(f"⭐ Priority added to cart: {product.name} - ${product.price}")

    def remove_product(self):
        """Remove the most recently added product (undo operation)."""
        if not self.is_empty():
            product = self.cart.pop()
            print(f"❌ Removed from cart: {product.name}")
            return product
        else:
            print("⚠️ Cart is empty. Cannot remove.")
            return None

    def remove_product_front(self):
        """Remove the first product from cart."""
        if not self.is_empty():
            product = self.cart.popleft()
            print(f"❌ Removed from front: {product.name}")
            return product
        else:
            print("⚠️ Cart is empty. Cannot remove.")
            return None

    def get_total_price(self):
        """Calculate total price of all items in cart."""
        total = sum(product.price * product.quantity for product in self.cart)
        return total

    def is_empty(self):
        """Check if cart is empty."""
        return len(self.cart) == 0

    def display_cart(self):
        """Display all items in the cart."""
        if self.is_empty():
            print("🛒 Your cart is empty.\n")
        else:
            print("\n🛒 SHOPPING CART:")
            print("-" * 70)
            total = 0
            for i, product in enumerate(self.cart, 1):
                item_total = product.price * product.quantity
                total += item_total
                print(f"{i}. {product.name} | Price: ${product.price} | Qty: {product.quantity} | Subtotal: ${item_total}")
            print("-" * 70)
            print(f"Total Price: ${total}")
            print("-" * 70 + "\n")

```

```python
# Example Usage
if __name__ == "__main__":
    print("=" * 70)
    print("🛒  SMART E-COMMERCE SHOPPING CART SYSTEM")
    print("=" * 70 + "\n")

    cart = ShoppingCart()

    # Create products
    p1 = Product(101, "Laptop", 999.99, 1)
    p2 = Product(102, "Wireless Mouse", 29.99, 2)
    p3 = Product(103, "USB-C Cable", 15.99, 3)
    p4 = Product(104, "Monitor", 299.99, 1)
    p5 = Product(105, "Keyboard", 79.99, 1)

    # Add products to cart
    print("📦 Adding products to cart:\n")
    cart.add_product(p1)
    cart.add_product(p2)
    cart.add_product(p3)
    cart.add_product_front(p4)  # Priority item
    cart.add_product(p5)

    # Display cart
    cart.display_cart()

    # Remove products (undo operation)
    print("🔄 Removing products (Undo operations):\n")
    cart.remove_product()
    cart.remove_product_front()

    # Display updated cart
    cart.display_cart()

    # Continue shopping
    print("📦 Adding more products:\n")
    p6 = Product(106, "Headphones", 149.99, 1)
    cart.add_product(p6)

    # Final cart display
    cart.display_cart()

    # Summary
    print(f"✅ Final Total: ${cart.get_total_price():.2f}")
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
======================================================================
🛍️ SMART E-COMMERCE SHOPPING CART SYSTEM
======================================================================

📦 Adding products to cart:

✅ Added to cart: Laptop - $999.99
✅ Added to cart: Wireless Mouse - $29.99
✅ Added to cart: USB-C Cable - $15.99
⭐ Priority added to cart: Monitor - $299.99
✅ Added to cart: Keyboard - $79.99

🛒 SHOPPING CART:
----------------------------------------------------------------------
1. Monitor | Price: $299.99 | Qty: 1 | Subtotal: $299.99
2. Laptop | Price: $999.99 | Qty: 1 | Subtotal: $999.99
3. Wireless Mouse | Price: $29.99 | Qty: 2 | Subtotal: $59.98
4. USB-C Cable | Price: $15.99 | Qty: 3 | Subtotal: $47.97
5. Keyboard | Price: $79.99 | Qty: 1 | Subtotal: $79.99
----------------------------------------------------------------------
Total Price: $1487.92
----------------------------------------------------------------------

🔄 Removing products (Undo operations):

❌ Removed from cart: Keyboard
❌ Removed from front: Monitor

🛒 SHOPPING CART:
----------------------------------------------------------------------
1. Laptop | Price: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Price: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Price: $15.99 | Qty: 3 | Subtotal: $47.97
----------------------------------------------------------------------
Total Price: $1107.94
----------------------------------------------------------------------

📦 Adding more products:
```

```
📦 Adding more products:

✅ Added to cart: Headphones - $149.99

🛒 SHOPPING CART:
----------------------------------------------------------------------
1. Laptop | Price: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Price: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Price: $15.99 | Qty: 3 | Subtotal: $47.97
4. Headphones | Price: $149.99 | Qty: 1 | Subtotal: $149.99
----------------------------------------------------------------------
Total Price: $1257.93
----------------------------------------------------------------------

✅ Final Total: $1257.93
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

<u>Task #13: Smart Logistics & Warehouse Management</u>

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity

5. Inventory Add/Remove Operations

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - o Stack
  - o Queue
  - o Priority Queue
  - o Linked List
  - o Binary Search Tree (BST)
  - o Graph
  - o Hash Table
  - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
"""
Smart Logistics & Warehouse Management
Scenario:
A logistics company wants to implement:
1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations
Student Task
• For each feature, select the most appropriate data structure from
the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2-3 sentences per feature.
• Implement one selected feature as a working Python program with
AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature
with comments and docstrings.
"""

"""
FEATURE MAPPING TABLE:
====================
1. Shipment Processing → Queue
    Justification: Shipments must be processed in the order they arrive for fair distribution.
    Queue's FIFO nature ensures shipments are handled chronologically without delays.

2. Urgent Shipment Priority → Priority Queue
    Justification: Urgent shipments need expedited handling regardless of arrival time.
    Priority queue allows sorting by urgency level to serve critical shipments first.

3. Product Lookup by Code → Hash Table
    Justification: Provides O(1) average lookup time using product code as key.
    Enables instant retrieval of product details, stock levels, and warehouse location.

4. Warehouse Network Connectivity → Graph
    Justification: Warehouses are nodes and shipping routes are edges in the network.
    Graph structure enables pathfinding and connectivity analysis between distribution centers.

5. Inventory Add/Remove Operations → Stack
    Justification: Uses LIFO principle for inventory management (last added items are picked first).
    Stack efficiently handles quick addition and removal of stock from warehouse shelves.
"""

# IMPLEMENTATION: Shipment Processing System using Queue

class Shipment:
    """Represents a shipment in the logistics system."""

    def __init__(self, shipment_id, destination, weight, status="Pending"):
        self.shipment_id = shipment_id
        self.destination = destination
        self.weight = weight
        self.status = status
```

```python
    def __repr__(self):
        return f"Shipment({self.shipment_id}, {self.destination}, {self.weight}kg, {self.status})"


class ShipmentQueue:
    """Queue-based shipment processing system."""

    def __init__(self):
        self.queue = []
        self.processed = []

    def enqueue_shipment(self, shipment):
        """Add a shipment to the processing queue."""
        self.queue.append(shipment)
        print(f"✅ Shipment {shipment.shipment_id} queued to {shipment.destination}")

    def process_shipment(self):
        """Process the next shipment in queue (FIFO)."""
        if not self.is_empty():
            shipment = self.queue.pop(0)
            shipment.status = "Processed"
            self.processed.append(shipment)
            print(f"📦 Processing: {shipment.shipment_id} to {shipment.destination}")
            return shipment
        else:
            print("⚠️ No shipments to process.")
            return None

    def peek_next_shipment(self):
        """View the next shipment without processing."""
        if not self.is_empty():
            shipment = self.queue[0]
            print(f"👀 Next shipment: {shipment.shipment_id} to {shipment.destination}")
            return shipment
        else:
            print("⚠️ No shipments in queue.")
            return None

    def is_empty(self):
        """Check if queue is empty."""
        return len(self.queue) == 0

    def display_queue(self):
        """Display all pending shipments."""
        if self.is_empty():
            print("📭 No pending shipments.\n")
        else:
            print("\n📋 PENDING SHIPMENTS QUEUE:")
            print("-" * 70)
            for i, shipment in enumerate(self.queue, 1):
                print(f"{i}. ID: {shipment.shipment_id} | Destination: {shipment.destination} | Weight: {shipment.weight}kg")
            print("-" * 70 + "\n")
```

```python

    def display_processed(self):
        """Display all processed shipments."""
        if not self.processed:
            print("📭 No processed shipments yet.\n")
        else:
            print("\n✅ PROCESSED SHIPMENTS:")
            print("-" * 70)
            for i, shipment in enumerate(self.processed, 1):
                print(f"{i}. ID: {shipment.shipment_id} | Destination: {shipment.destination} | Weight: {shipment.weight}kg")
            print("-" * 70 + "\n")


# Example Usage
if __name__ == "__main__":
    print("=" * 70)
    print("🚚 SMART LOGISTICS SHIPMENT PROCESSING SYSTEM")
    print("=" * 70 + "\n")

    system = ShipmentQueue()

    # Create and add shipments
    print("📦 Adding shipments to queue:\n")
    s1 = Shipment("SHP-001", "New York", 50)
    s2 = Shipment("SHP-002", "Los Angeles", 75)
    s3 = Shipment("SHP-003", "Chicago", 60)
    s4 = Shipment("SHP-004", "Houston", 45)
    s5 = Shipment("SHP-005", "Miami", 55)

    system.enqueue_shipment(s1)
    system.enqueue_shipment(s2)
    system.enqueue_shipment(s3)
    system.enqueue_shipment(s4)
    system.enqueue_shipment(s5)

    # Display pending queue
    system.display_queue()

    # Peek next shipment
    print("🔍 Checking next shipment:\n")
    system.peek_next_shipment()

    # Process shipments
    print("\n🚀 PROCESSING SHIPMENTS:\n")
    system.process_shipment()
    system.process_shipment()
    system.process_shipment()

    # Display updated queue
    system.display_queue()

    # Display processed shipments
    system.display_processed()

    # Process remaining shipments
    print("Processing remaining shipments:\n")
    system.process_shipment()
    system.process_shipment()

    system.display_queue()
    system.display_processed()

```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
======================================================================
🚚 SMART LOGISTICS SHIPMENT PROCESSING SYSTEM
======================================================================

📦 Adding shipments to queue:

✅ Shipment SHP-001 queued to New York
✅ Shipment SHP-002 queued to Los Angeles
✅ Shipment SHP-003 queued to Chicago
✅ Shipment SHP-004 queued to Houston
✅ Shipment SHP-005 queued to Miami

📋 PENDING SHIPMENTS QUEUE:
----------------------------------------------------------------------
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
4. ID: SHP-004 | Destination: Houston | Weight: 45kg
5. ID: SHP-005 | Destination: Miami | Weight: 55kg
----------------------------------------------------------------------

🔍 Checking next shipment:

👀 Next shipment: SHP-001 to New York

🚀 PROCESSING SHIPMENTS:

📦 Processing: SHP-001 to New York
📦 Processing: SHP-002 to Los Angeles
📦 Processing: SHP-003 to Chicago

📋 PENDING SHIPMENTS QUEUE:
----------------------------------------------------------------------
1. ID: SHP-004 | Destination: Houston | Weight: 45kg
2. ID: SHP-005 | Destination: Miami | Weight: 55kg
----------------------------------------------------------------------


✅ PROCESSED SHIPMENTS:
```

```
✅ PROCESSED SHIPMENTS:
----------------------------------------------------------------------
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
----------------------------------------------------------------------

Processing remaining shipments:

📦 Processing: SHP-004 to Houston
📦 Processing: SHP-005 to Miami
📭 No pending shipments.


✅ PROCESSED SHIPMENTS:
----------------------------------------------------------------------
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
4. ID: SHP-004 | Destination: Houston | Weight: 45kg
5. ID: SHP-005 | Destination: Miami | Weight: 55kg
----------------------------------------------------------------------
```

<u>Task Description #14: Smart Banking Management System – Data</u>

Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.

2. Loan Approval Priority Handling – High-value or urgent loans processed first.

3. Account Information Lookup – Fast retrieval of customer details using Account Number.

4. Transaction History Management – Maintain chronological

transaction records.

5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
"""
FEATURE MAPPING TABLE:
======================
1. Customer Service Token System → Queue
    Justification: Customers must be served in FIFO order based on token arrival time.
    Queue ensures fair service and maintains chronological order without favoritism.

2. Loan Approval Priority Handling → Priority Queue
    Justification: High-value or urgent loans need expedited processing regardless of arrival time.
    Priority queue automatically prioritizes loans by importance, processing critical applications first.

3. Account Information Lookup → Hash Table
    Justification: Provides O(1) average lookup time using Account Number as key.
    Enables instant retrieval of customer details, balance, and account status.

4. Transaction History Management → Linked List
    Justification: Maintains chronological order of transactions with easy insertion/deletion.
    Linked list efficiently stores transaction records without requiring contiguous memory.

5. ATM Network Connectivity → Graph
    Justification: ATMs are nodes and connections between cities are edges in the network.
    Graph enables pathfinding and connectivity analysis for ATM availability across regions.
"""

# IMPLEMENTATION: Customer Service Token System using Queue

class Token:
    """Represents a customer service token."""

    def __init__(self, token_id, customer_name, service_type):
        self.token_id = token_id
        self.customer_name = customer_name
        self.service_type = service_type
        self.status = "Waiting"

    def __repr__(self):
        return f"Token({self.token_id}, {self.customer_name}, {self.service_type})"


class BankServiceQueue:
    """Manages customer service tokens in FIFO order."""

    def __init__(self):
        self.queue = []
        self.served_tokens = []

    def issue_token(self, token_id, customer_name, service_type):
        """Issue a new service token to a customer."""
        token = Token(token_id, customer_name, service_type)
        self.queue.append(token)
        print(f"✅ Token #{token_id} issued to {customer_name} for {service_type}")

    def call_next_customer(self):
        """Call the next customer in queue."""
        if not self.is_empty():
            token = self.queue.pop(0)
            token.status = "Served"
            self.served_tokens.append(token)
            print(f"📢 Calling: Token #{token.token_id} - {token.customer_name} ({token.service_type})")
            return token
        else:
            print("⚠️ No customers in queue.")
            return None
```

```python
            return None

    def peek_next_customer(self):
        """Check who is the next customer without removing from queue."""
        if not self.is_empty():
            token = self.queue[0]
            print(f"👁️ Next customer: Token #{token.token_id} - {token.customer_name}")
            return token
        else:
            print("⚠️ Queue is empty.")
            return None

    def is_empty(self):
        """Check if queue is empty."""
        return len(self.queue) == 0

    def display_waiting_queue(self):
        """Display all customers waiting in queue."""
        if self.is_empty():
            print("📭 No customers waiting.\n")
        else:
            print("\n📋 CUSTOMERS WAITING IN QUEUE:")
            print("-" * 70)
            for i, token in enumerate(self.queue, 1):
                print(f"{i}. Token #{token.token_id} | {token.customer_name} | {token.service_type}")
            print("-" * 70 + "\n")

    def display_served_customers(self):
        """Display all served customers."""
        if not self.served_tokens:
            print("📭 No customers served yet.\n")
        else:
            print("\n✅ SERVED CUSTOMERS:")
            print("-" * 70)
            for i, token in enumerate(self.served_tokens, 1):
                print(f"{i}. Token #{token.token_id} | {token.customer_name} | {token.service_type}")
            print("-" * 70 + "\n")


# Example Usage
if __name__ == "__main__":
    print("=" * 70)
    print("🏦 SMART BANKING CUSTOMER SERVICE SYSTEM")
    print("=" * 70 + "\n")

    bank = BankServiceQueue()

    # Issue tokens to customers
    print("📝 Issuing service tokens:\n")
    bank.issue_token(101, "Alice Johnson", "Deposit")
    bank.issue_token(102, "Bob Smith", "Withdrawal")
    bank.issue_token(103, "Carol White", "Loan Application")
    bank.issue_token(104, "David Brown", "Account Opening")
    bank.issue_token(105, "Eve Davis", "Balance Inquiry")

    # Display waiting queue
    bank.display_waiting_queue()

    # Check next customer
    print("🔍 Checking next customer:\n")
    bank.peek_next_customer()

    # Serve customers
    print("\n📞 SERVING CUSTOMERS:\n")
    bank.call_next_customer()
    bank.call_next_customer()
    bank.call_next_customer()

    # Display updated queue
    bank.display_waiting_queue()

    # Display served customers
    bank.display_served_customers()

    # Serve remaining customers
    print("Serving remaining customers:\n")
    bank.call_next_customer()
    bank.call_next_customer()

    bank.display_waiting_queue()
    bank.display_served_customers()
```

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
======================================================================
🏦 SMART BANKING CUSTOMER SERVICE SYSTEM
======================================================================

🎫 Issuing service tokens:

✅ Token #101 issued to Alice Johnson for Deposit
✅ Token #102 issued to Bob Smith for Withdrawal
✅ Token #103 issued to Carol White for Loan Application
✅ Token #104 issued to David Brown for Account Opening
✅ Token #105 issued to Eve Davis for Balance Inquiry

👥 CUSTOMERS WAITING IN QUEUE:
----------------------------------------------------------------------
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
4. Token #104 | David Brown | Account Opening
5. Token #105 | Eve Davis | Balance Inquiry
----------------------------------------------------------------------

🔍 Checking next customer:

👀 Next customer: Token #101 - Alice Johnson

📞 SERVING CUSTOMERS:

📢 Calling: Token #101 - Alice Johnson (Deposit)
📢 Calling: Token #102 - Bob Smith (Withdrawal)
📢 Calling: Token #103 - Carol White (Loan Application)

👥 CUSTOMERS WAITING IN QUEUE:
----------------------------------------------------------------------
1. Token #104 | David Brown | Account Opening
2. Token #105 | Eve Davis | Balance Inquiry
----------------------------------------------------------------------
```

```
✅ SERVED CUSTOMERS:
----------------------------------------------------------------------
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
----------------------------------------------------------------------

Serving remaining customers:

📢 Calling: Token #104 - David Brown (Account Opening)
📢 Calling: Token #105 - Eve Davis (Balance Inquiry)
📭 No customers waiting.


✅ SERVED CUSTOMERS:
----------------------------------------------------------------------
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
4. Token #104 | David Brown | Account Opening
5. Token #105 | Eve Davis | Balance Inquiry
----------------------------------------------------------------------

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

<u>Task Description #15: Online Learning Management System – Data</u>

Structure Selection

An online education platform wants to develop a Learning Management

System that handles:

1. Student Login Authentication – Quick validation of user

credentials.

2. Assignment Submission Queue – Submissions processed in order

of upload.

3. Top Performer Ranking – Rank students based on scores.

4. Course Prerequisite Structure – Represent subject dependencies.

5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

• Select the most appropriate data structure for each feature from the given list.

• Justify your selection in 2–3 sentences per feature.

• Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

• Feature → Data Structure → Justification table.

• Functional Python program with proper documentation.

Code:

```
FEATURE MAPPING TABLE:
=====================
1. Student Login Authentication → Hash Table
    Justification: Provides O(1) average lookup time using username/email as key.
    Enables instant credential validation and fast user authentication without delays.

2. Assignment Submission Queue → Queue
    Justification: Submissions must be processed in FIFO order based on upload time.
    Queue ensures fair evaluation and maintains chronological order of submissions.

3. Top Performer Ranking → Priority Queue
    Justification: Students ranked by scores with highest performers prioritized.
    Priority queue automatically maintains sorted order for quick access to top performers.

4. Course Prerequisite Structure → Graph
    Justification: Courses are nodes and prerequisites are directed edges between them.
    Graph enables validation of course dependencies and pathfinding for course sequences.

5. Undo Feature in Online Editor → Stack
    Justification: Uses LIFO principle to reverse actions in reverse order of execution.
    Stack efficiently stores action history and allows quick undo operations.
"""

# IMPLEMENTATION: Assignment Submission Queue using Queue

class Assignment:

    def __init__(self, submission_id, student_name, assignment_name, submission_time, score=None):
        self.submission_id = submission_id
        self.student_name = student_name
        self.assignment_name = assignment_name
        self.submission_time = submission_time
        self.score = score
        self.status = "Pending"

    def __repr__(self):
        return f"Assignment({self.submission_id}, {self.student_name}, {self.assignment_name})"


class AssignmentSubmissionQueue:

    def __init__(self):
        self.queue = []
        self.evaluated = []

```

```python
    def submit_assignment(self, submission_id, student_name, assignment_name, submission_time):
        assignment = Assignment(submission_id, student_name, assignment_name, submission_time)
        self.queue.append(assignment)
        print(f"✅ Assignment submitted: {student_name} - {assignment_name} at {submission_time}")

    def evaluate_next_assignment(self, score):
        if not self.is_empty():
            assignment = self.queue.pop(0)
            assignment.status = "Evaluated"
            assignment.score = score
            self.evaluated.append(assignment)
            print(f"📝 Evaluated: {assignment.student_name}'s {assignment.assignment_name} (Score: {score}/100)")
            return assignment
        else:
            print("⚠️ No assignments to evaluate.")
            return None

    def peek_next_assignment(self):
        if not self.is_empty():
            assignment = self.queue[0]
            print(f"👀 Next assignment: {assignment.student_name} - {assignment.assignment_name}")
            return assignment
        else:
            print("⚠️ Queue is empty.")
            return None

    def is_empty(self):
        return len(self.queue) == 0

    def display_pending_queue(self):
        if self.is_empty():
            print("📭 No pending assignments.\n")
        else:
            print("\n📋 PENDING ASSIGNMENTS QUEUE:")
            print("-" * 80)
            for i, assignment in enumerate(self.queue, 1):
                print(f"{i}. ID: {assignment.submission_id} | {assignment.student_name} | {assignment.assignment_name} | {assignment.submission_time}")
            print("-" * 80 + "\n")

    def display_evaluated_assignments(self):
        if not self.evaluated:
            print("📭 No evaluated assignments yet.\n")
        else:
            print("\n✅ EVALUATED ASSIGNMENTS:")
            print("-" * 80)
            for i, assignment in enumerate(self.evaluated, 1):
                print(f"{i}. ID: {assignment.submission_id} | {assignment.student_name} | {assignment.assignment_name} | Score: {assignment.score}/100")
            print("-" * 80 + "\n")
```

```python
# Example Usage
if __name__ == "__main__":
    print("=" * 80)
    print("🎓 ONLINE LEARNING MANAGEMENT SYSTEM - ASSIGNMENT SUBMISSION QUEUE")
    print("=" * 80 + "\n")

    system = AssignmentSubmissionQueue()

    # Submit assignments
    print("📤 Submitting assignments:\n")
    system.submit_assignment(1001, "Alice Johnson", "Python Basics", "09:15 AM")
    system.submit_assignment(1002, "Bob Smith", "Data Structures", "09:30 AM")
    system.submit_assignment(1003, "Carol White", "Python Basics", "09:45 AM")
    system.submit_assignment(1004, "David Brown", "Web Development", "10:00 AM")
    system.submit_assignment(1005, "Eve Davis", "Data Structures", "10:15 AM")

    # Display pending queue
    system.display_pending_queue()

    # Check next assignment
    print("🔍 Checking next assignment to evaluate:\n")
    system.peek_next_assignment()

    # Evaluate assignments
    print("\n📝 EVALUATING ASSIGNMENTS:\n")
    system.evaluate_next_assignment(92)
    system.evaluate_next_assignment(85)
    system.evaluate_next_assignment(88)

    # Display updated queue
    system.display_pending_queue()

    # Display evaluated assignments
    system.display_evaluated_assignments()

    # Evaluate remaining assignments
    print("Evaluating remaining assignments:\n")
    system.evaluate_next_assignment(95)
    system.evaluate_next_assignment(78)

    system.display_pending_queue()
    system.display_evaluated_assignments()
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
================================================================================
🎓 ONLINE LEARNING MANAGEMENT SYSTEM - ASSIGNMENT SUBMISSION QUEUE
================================================================================

📤 Submitting assignments:

✅ Assignment submitted: Alice Johnson - Python Basics at 09:15 AM
✅ Assignment submitted: Bob Smith - Data Structures at 09:30 AM
✅ Assignment submitted: Carol White - Python Basics at 09:45 AM
✅ Assignment submitted: David Brown - Web Development at 10:00 AM
✅ Assignment submitted: Eve Davis - Data Structures at 10:15 AM

📋 PENDING ASSIGNMENTS QUEUE:
--------------------------------------------------------------------------------
1. ID: 1001 | Alice Johnson | Python Basics | 09:15 AM
2. ID: 1002 | Bob Smith | Data Structures | 09:30 AM
3. ID: 1003 | Carol White | Python Basics | 09:45 AM
4. ID: 1004 | David Brown | Web Development | 10:00 AM
5. ID: 1005 | Eve Davis | Data Structures | 10:15 AM
--------------------------------------------------------------------------------

🔍 Checking next assignment to evaluate:

👀 Next assignment: Alice Johnson - Python Basics

📝 EVALUATING ASSIGNMENTS:

📝 Evaluated: Alice Johnson's Python Basics (Score: 92/100)
📝 Evaluated: Bob Smith's Data Structures (Score: 85/100)
📝 Evaluated: Carol White's Python Basics (Score: 88/100)

📋 PENDING ASSIGNMENTS QUEUE:
--------------------------------------------------------------------------------
1. ID: 1004 | David Brown | Web Development | 10:00 AM
2. ID: 1005 | Eve Davis | Data Structures | 10:15 AM
--------------------------------------------------------------------------------
```

```
✅ EVALUATED ASSIGNMENTS:
--------------------------------------------------------------------------------
1. ID: 1001 | Alice Johnson | Python Basics | Score: 92/100
2. ID: 1002 | Bob Smith | Data Structures | Score: 85/100
3. ID: 1003 | Carol White | Python Basics | Score: 88/100
--------------------------------------------------------------------------------

Evaluating remaining assignments:

📝 Evaluated: David Brown's Web Development (Score: 95/100)
📝 Evaluated: Eve Davis's Data Structures (Score: 78/100)
📭 No pending assignments.


✅ EVALUATED ASSIGNMENTS:
--------------------------------------------------------------------------------
1. ID: 1001 | Alice Johnson | Python Basics | Score: 92/100
2. ID: 1002 | Bob Smith | Data Structures | Score: 85/100
3. ID: 1003 | Carol White | Python Basics | Score: 88/100
4. ID: 1004 | David Brown | Web Development | Score: 95/100
5. ID: 1005 | Eve Davis | Data Structures | Score: 78/100
--------------------------------------------------------------------------------
```

<u>Task Description #16: Smart Airport Management System – Data</u>

Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of

arrival.

2. Emergency Landing Priority – Aircraft in emergency get priority.

3. Flight Information Lookup – Fast search using Flight ID.

4. Air Route Connectivity – Airports connected via routes.

5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

Code:

```python
"""
Smart Airport Management System — Data Structure Challenge

An airport authority plans to implement:
1. Passenger Check-In Queue — Process passengers in order of arrival.
2. Emergency Landing Priority — Aircraft in emergency get priority.
3. Flight Information Lookup — Fast search using Flight ID.
4. Air Route Connectivity — Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) — Overhead luggage management simulation.

Student Task:
• Choose suitable data structures from the provided list.
• Provide 2–3 sentence justification per feature.
• Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:
• Mapping table.
• Working Python implementation.
"""

# FEATURE MAPPING TABLE
FEATURE_MAPPING = [
    (
        "Passenger Check-In Queue",
        "Queue",
        "Passengers must be processed in FIFO order based on arrival time. "
        "Queue ensures fair service and maintains chronological processing without delays."
    ),
    (
        "Emergency Landing Priority",
        "Priority Queue",
        "Aircraft in emergency situations require immediate landing priority. "
        "Priority queue sorts by urgency level, ensuring critical aircraft land first."
    ),
    (
        "Flight Information Lookup",
        "Hash Table",
        "Provides O(1) average lookup time using Flight ID as key. "
        "Enables instant retrieval of flight details like schedule, gate, and status."
    ),
    (
        "Air Route Connectivity",
        "Graph",
        "Airports are nodes and flight routes are edges connecting them. "
        "Graph structure enables pathfinding and connectivity analysis for route planning."
    ),
    (
        "Boarding Process (Cabin Storage)",
```

```python
def print_feature_mapping_table():
    """Display the feature mapping table with justifications."""
    print("\n" + "=" * 130)
    print("SMART AIRPORT MANAGEMENT SYSTEM - FEATURE MAPPING TABLE")
    print("=" * 130)
    print(f"{'Feature':<35} {'Data Structure':<18} {'Justification':<75}")
    print("-" * 130)
    for feature, structure, justification in FEATURE_MAPPING:
        print(f"{feature:<35} {structure:<18} {justification:<75}")
    print("=" * 130 + "\n")


# IMPLEMENTATION: Passenger Check-In Queue using Queue Data Structure

class Passenger:
    """Represents a passenger with check-in details."""

    def __init__(self, passenger_id, name, flight_id, arrival_time):
        """
        Initialize a passenger object.

        Args:
            passenger_id: Unique identifier for the passenger
            name: Name of the passenger
            flight_id: Flight ID for the passenger's booking
            arrival_time: Time of arrival at check-in counter
        """
        self.passenger_id = passenger_id
        self.name = name
        self.flight_id = flight_id
        self.arrival_time = arrival_time
        self.status = "Waiting"

    def __repr__(self):
        return f"Passenger({self.passenger_id}, {self.name}, Flight: {self.flight_id}, Time: {self.arrival_time})"


class PassengerCheckInQueue:
    """Manages passenger check-in queue using FIFO principle."""

    def __init__(self):
        """Initialize the check-in queue and tracking lists."""
        self.queue = []
        self.checked_in = []

    def add_passenger(self, passenger_id, name, flight_id, arrival_time):
        """
        Add a passenger to the check-in queue.

        Args:
            passenger_id: Unique passenger identifier
```

```python
            passenger_id: Unique passenger identifier
            name: Passenger name
            flight_id: Flight ID
            arrival_time: Arrival time at counter
        """
        passenger = Passenger(passenger_id, name, flight_id, arrival_time)
        self.queue.append(passenger)
        print(f"✅ Passenger added to queue: {name} (ID: {passenger_id}) - Flight {flight_id}")

    def process_check_in(self):
        """
        Process the next passenger in queue (FIFO).

        Returns:
            Passenger object if processed, None if queue is empty
```

Click to add a breakpoint

```python
        if not self.is_empty():
            passenger = self.queue.pop(0)
            passenger.status = "Checked-In"
            self.checked_in.append(passenger)
            print(f"✔  Checked-In: {passenger.name} (Flight: {passenger.flight_id})")
            return passenger
        else:
            print("⚠  No passengers in queue.")
            return None

    def peek_next_passenger(self):
        """
        View the next passenger without removing from queue.

        Returns:
            Next passenger object if available, None if queue is empty
        """
        if not self.is_empty():
            passenger = self.queue[0]
            print(f"👀 Next passenger: {passenger.name} (Flight: {passenger.flight_id})")
            return passenger
        else:
            print("⚠  Queue is empty.")
            return None

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.queue) == 0
```

```python
class PassengerCheckInQueue:
    def display_waiting_passengers(self):
        """Display all passengers currently waiting in the queue."""
        if self.is_empty():
            print("📭 No passengers waiting.\n")
        else:
            print("\n📋 PASSENGER CHECK-IN QUEUE (FIFO):")
            print("-" * 100)
            for i, passenger in enumerate(self.queue, 1):
                print(f"{i}. ID: {passenger.passenger_id:<6} | Name: {passenger.name:<20} | Flight: {passenger.flight_id:<8} | Arrival: {passenger.arrival_time}")
            print("-" * 100 + "\n")

    def display_checked_in_passengers(self):
        """Display all passengers who have been checked in."""
        if not self.checked_in:
            print("📭 No passengers checked-in yet.\n")
        else:
            print("\n✅ CHECKED-IN PASSENGERS:")
            print("-" * 100)
            for i, passenger in enumerate(self.checked_in, 1):
                print(f"{i}. ID: {passenger.passenger_id:<6} | Name: {passenger.name:<20} | Flight: {passenger.flight_id:<8} | Status: {passenger.status}")
            print("-" * 100 + "\n")


# Example Usage
if __name__ == "__main__":
    # Display feature mapping table
    print_feature_mapping_table()

    print("=" * 100)
    print("🛫 SMART AIRPORT MANAGEMENT SYSTEM - PASSENGER CHECK-IN QUEUE")
    print("=" * 100 + "\n")

    # Create check-in system
    system = PassengerCheckInQueue()

    # Add passengers to queue
    print("📝 Adding passengers to check-in queue:\n")
    system.add_passenger(101, "Alice Johnson", "AA-101", "08:00 AM")
    system.add_passenger(102, "Bob Smith", "UA-205", "08:05 AM")
    system.add_passenger(103, "Carol White", "DL-310", "08:10 AM")
    system.add_passenger(104, "David Brown", "AA-101", "08:15 AM")
    system.add_passenger(105, "Eve Davis", "UA-205", "08:20 AM")
    system.add_passenger(106, "Frank Miller", "SW-415", "08:25 AM")

    # Display waiting passengers
    system.display_waiting_passengers()

    # Peek at next passenger
    print("🔍 Checking next passenger:\n")
```

assg_11.py > ...

```python
    # Display waiting passengers
    system.display_waiting_passengers()

    # Peek at next passenger
    print("🔍 Checking next passenger:\n")
    system.peek_next_passenger()
    print(f"\n📊 Queue Length: {system.get_queue_length()} passengers waiting\n")

    # Process check-ins
    print("✈️ PROCESSING CHECK-INS:\n")
    system.process_check_in()
    system.process_check_in()
    system.process_check_in()

    # Display updated queue
    system.display_waiting_passengers()
    print(f"📊 Remaining Queue Length: {system.get_queue_length()} passengers\n")

    # Display checked-in passengers
    system.display_checked_in_passengers()

    # Continue processing
    print("Continuing to process remaining passengers:\n")
    system.process_check_in()
    system.process_check_in()
    system.process_check_in()

    # Final displays
    system.display_waiting_passengers()
    system.display_checked_in_passengers()

    print("✈️  All passengers have been processed!")
```

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py

================================================================================================================================
SMART AIRPORT MANAGEMENT SYSTEM - FEATURE MAPPING TABLE
================================================================================================================================
Feature                          Data Structure     Justification
--------------------------------------------------------------------------------------------------------------------------------
Passenger Check-In Queue         Queue              Passengers must be processed in FIFO order based on arrival time. Queue ensures fair service and maintains
 chronological processing without delays.
Emergency Landing Priority       Priority Queue     Aircraft in emergency situations require immediate landing priority. Priority queue sorts by urgency level
, ensuring critical aircraft land first.
Flight Information Lookup        Hash Table         Provides O(1) average lookup time using Flight ID as key. Enables instant retrieval of flight details like
 schedule, gate, and status.
Air Route Connectivity           Graph              Airports are nodes and flight routes are edges connecting them. Graph structure enables pathfinding and co
nnectivity analysis for route planning.
Boarding Process (Cabin Storage) Stack              Uses LIFO principle for overhead luggage management. Stack efficiently handles last-loaded items being ret
rieved first during deplaning.
================================================================================================================================


==============================================================================================
🛫 SMART AIRPORT MANAGEMENT SYSTEM - PASSENGER CHECK-IN QUEUE
==============================================================================================

📋 Adding passengers to check-in queue:

✅ Passenger added to queue: Alice Johnson (ID: 101) - Flight AA-101
✅ Passenger added to queue: Bob Smith (ID: 102) - Flight UA-205
✅ Passenger added to queue: Carol White (ID: 103) - Flight DL-310
✅ Passenger added to queue: David Brown (ID: 104) - Flight AA-101
✅ Passenger added to queue: Eve Davis (ID: 105) - Flight UA-205
✅ Passenger added to queue: Frank Miller (ID: 106) - Flight SW-415

🛫 PASSENGER CHECK-IN QUEUE (FIFO):
----------------------------------------------------------------------------------------------
1. ID: 101    | Name: Alice Johnson        | Flight: AA-101   | Arrival: 08:00 AM
2. ID: 102    | Name: Bob Smith            | Flight: UA-205   | Arrival: 08:05 AM
3. ID: 103    | Name: Carol White          | Flight: DL-310   | Arrival: 08:10 AM
4. ID: 104    | Name: David Brown          | Flight: AA-101   | Arrival: 08:15 AM
5. ID: 105    | Name: Eve Davis            | Flight: UA-205   | Arrival: 08:20 AM
```

```
✅ CHECKED-IN PASSENGERS:
----------------------------------------------------------------------------------------------
1. ID: 101    | Name: Alice Johnson        | Flight: AA-101   | Status: Checked-In
2. ID: 102    | Name: Bob Smith            | Flight: UA-205   | Status: Checked-In
3. ID: 103    | Name: Carol White          | Flight: DL-310   | Status: Checked-In
----------------------------------------------------------------------------------------------

Continuing to process remaining passengers:

✔ Checked-In: David Brown (Flight: AA-101)
✔ Checked-In: Eve Davis (Flight: UA-205)
✔ Checked-In: Frank Miller (Flight: SW-415)
📭 No passengers waiting.


✅ CHECKED-IN PASSENGERS:
----------------------------------------------------------------------------------------------
1. ID: 101    | Name: Alice Johnson        | Flight: AA-101   | Status: Checked-In
2. ID: 102    | Name: Bob Smith            | Flight: UA-205   | Status: Checked-In
3. ID: 103    | Name: Carol White          | Flight: DL-310   | Status: Checked-In
4. ID: 104    | Name: David Brown          | Flight: AA-101   | Status: Checked-In
5. ID: 105    | Name: Eve Davis            | Flight: UA-205   | Status: Checked-In
6. ID: 106    | Name: Frank Miller         | Flight: SW-415   | Status: Checked-In
----------------------------------------------------------------------------------------------

🛫 All passengers have been processed!
```

<u>Task Description #17: Smart Social Media Platform – Data Structure</u>

Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.

2. Trending Hashtags Tracker – Rank hashtags by usage frequency.

3. User Profile Search – Quick lookup using username.

4. Friend Network Representation – Represent user connections.

5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

• Select appropriate data structures from the list.

• Justify your choices clearly.

• Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

• Table mapping features to data structures with justification.

• Working Python code with docstrings and comments.

Code:

```python
FEATURE MAPPING TABLE:
=====================
1. News Feed Generation → Linked List
    Justification: Maintains chronological order of posts with easy insertion at the beginning.
    Linked list efficiently handles dynamic post addition without requiring contiguous memory.

2. Trending Hashtags Tracker → Priority Queue
    Justification: Hashtags ranked by usage frequency with most popular appearing first.
    Priority queue automatically maintains sorted order for quick access to trending topics.

3. User Profile Search → Hash Table
    Justification: Provides O(1) average lookup time using username as key.
    Enables instant retrieval of user profiles and account information.

4. Friend Network Representation → Graph
    Justification: Users are nodes and friend connections are edges in the network.
    Graph structure enables pathfinding and connectivity analysis for friend recommendations.

5. Message Undo Feature → Stack
    Justification: Uses LIFO principle to reverse messages in reverse order of sending.
    Stack efficiently stores message history and allows quick undo operations.
"""

# IMPLEMENTATION: Trending Hashtags Tracker using Priority Queue



class Hashtag:
    """Represents a hashtag with its usage count."""

    def __init__(self, tag_name, usage_count=1):
        self.tag_name = tag_name
        self.usage_count = usage_count

    def __lt__(self, other):
        """Compare hashtags by usage count (higher count = higher priority)."""
        return self.usage_count > other.usage_count

    def __repr__(self):
        return f"#{self.tag_name} ({self.usage_count})"


class TrendingHashtagTracker:
    """
    Manages trending hashtags using a Priority Queue (min-heap).
    Higher usage count = higher priority = more trending.
    """

    def __init__(self):
        self.heap = []
        self.hashtag_dict = {}

```

```python
    def add_hashtag(self, tag_name):
        """
        Add a hashtag or increment its usage count.

        Args:
            tag_name: Name of the hashtag (without #)
        """
        if tag_name in self.hashtag_dict:
            self.hashtag_dict[tag_name] += 1
            print(f"📈 Updated #{tag_name} - Usage count: {self.hashtag_dict[tag_name]}")
        else:
            self.hashtag_dict[tag_name] = 1
            print(f"✅ Added new hashtag #{tag_name}")

    def get_trending_hashtags(self, limit=5):
        """
        Retrieve top trending hashtags.

        Args:
            limit: Number of top hashtags to return

        Returns:
            List of top trending hashtags
        """
        heap = []
        for tag_name, count in self.hashtag_dict.items():
            heapq.heappush(heap, (-count, tag_name))

        trending = []
        for _ in range(min(limit, len(heap))):
            count, tag_name = heapq.heappop(heap)
            trending.append((tag_name, -count))

        return trending

    def display_top_trending(self, limit=5):
        """Display top trending hashtags."""
        trending = self.get_trending_hashtags(limit)

        if not trending:
            print("📭 No hashtags yet.\n")
        else:
            print(f"\n🔥 TOP {limit} TRENDING HASHTAGS:")
            print("-" * 60)
            for i, (tag_name, count) in enumerate(trending, 1):
                print(f"{i}. #{tag_name:<20} | Usage: {count:>3} times | 📊 {'▇' * (count // 5)}")
            print("-" * 60 + "\n")
```

```python
    def display_all_hashtags(self):
        """Display all hashtags sorted by usage count."""
        if not self.hashtag_dict:
            print("📭 No hashtags yet.\n")
        else:
            sorted_tags = sorted(self.hashtag_dict.items(), key=lambda x: x[1], reverse=True)
            print("\n📋 ALL HASHTAGS (Sorted by Usage):")
            print("-" * 60)
            for i, (tag_name, count) in enumerate(sorted_tags, 1):
                print(f"{i}. #{tag_name:<20} | Usage: {count:>3} times")
            print("-" * 60 + "\n")

    def remove_hashtag(self, tag_name):
        """Remove a hashtag."""
        if tag_name in self.hashtag_dict:
            del self.hashtag_dict[tag_name]
            print(f"❌ Removed hashtag #{tag_name}")
        else:
            print(f"⚠️  Hashtag #{tag_name} not found.")
```

```python
# Example Usage
if __name__ == "__main__":
    print("=" * 60)
    print("📊 SMART SOCIAL MEDIA PLATFORM - TRENDING HASHTAGS TRACKER")
    print("=" * 60 + "\n")

    tracker = TrendingHashtagTracker()

    # Add hashtags with multiple uses
    print("📝 Adding hashtags from posts:\n")
    tracker.add_hashtag("Python")
    tracker.add_hashtag("DataStructures")
    tracker.add_hashtag("Python")
    tracker.add_hashtag("Coding")
    tracker.add_hashtag("Python")
    tracker.add_hashtag("WebDevelopment")
    tracker.add_hashtag("DataStructures")
    tracker.add_hashtag("Python")
    tracker.add_hashtag("AI")
    tracker.add_hashtag("Coding")
    tracker.add_hashtag("Python")
    tracker.add_hashtag("AI")

    # Display all hashtags
    tracker.display_all_hashtags()

    # Display top trending
    print("🏆 Checking trending hashtags:\n")
    tracker.display_top_trending(5)

    # Add more activity to a hashtag
    print("📤 More posts using hashtags:\n")
    tracker.add_hashtag("AI")
    tracker.add_hashtag("AI")
    tracker.add_hashtag("AI")

    # Display updated trending
    tracker.display_top_trending(5)

    # Remove a hashtag
    print("🗑️  Removing hashtag:\n")
    tracker.remove_hashtag("WebDevelopment")

    # Final display
    tracker.display_all_hashtags()
```

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_11.py
============================================================
📱 SMART SOCIAL MEDIA PLATFORM - TRENDING HASHTAGS TRACKER
============================================================

📝 Adding hashtags from posts:

✅ Added new hashtag #Python
✅ Added new hashtag #DataStructures
📈 Updated #Python - Usage count: 2
✅ Added new hashtag #Coding
📈 Updated #Python - Usage count: 3
✅ Added new hashtag #WebDevelopment
📈 Updated #DataStructures - Usage count: 2
📈 Updated #Python - Usage count: 4
✅ Added new hashtag #AI
📈 Updated #Coding - Usage count: 2
📈 Updated #Python - Usage count: 5
📈 Updated #AI - Usage count: 2

📋 ALL HASHTAGS (Sorted by Usage):
------------------------------------------------------------
1. #Python              | Usage:   5 times
2. #DataStructures      | Usage:   2 times
3. #Coding              | Usage:   2 times
4. #AI                  | Usage:   2 times
5. #WebDevelopment      | Usage:   1 times
------------------------------------------------------------

🏆 Checking trending hashtags:


🔥 TOP 5 TRENDING HASHTAGS:
------------------------------------------------------------
1. #Python              | Usage:   5 times | 📊
2. #AI                  | Usage:   2 times | 📊
3. #Coding              | Usage:   2 times | 📊
4. #DataStructures      | Usage:   2 times | 📊
5. #WebDevelopment      | Usage:   1 times | 📊
```

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment

2. Medical Emergency Handling

3. Student Record Retrieval

4. Hostel Block Connectivity

5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

Code:

```python
"""
FEATURE MAPPING TABLE:
======================
1. Room Allotment → Queue
    Justification: Students are assigned rooms in the order they apply.
    Queue ensures fair allocation and maintains chronological order without bias.

2. Medical Emergency Handling → Priority Queue (implemented without heapq)
    Justification: Critical patients need expedited attention regardless of arrival time.
    Priority queue sorts by severity level to serve emergencies in order of urgency.

3. Student Record Retrieval → Hash Table
    Justification: Provides O(1) average lookup time using roll number as key.
    Enables instant retrieval of student details and academic records.

4. Hostel Block Connectivity → Graph
    Justification: Blocks are nodes and connections between blocks are edges.
    Graph structure enables pathfinding and connectivity analysis for navigation.

5. Visitor Entry Log → Linked List
    Justification: Maintains chronological order of visitor entries with easy insertion.
    Linked list efficiently handles dynamic logging without requiring contiguous memory.
"""

# IMPLEMENTATION: Medical Emergency Handling using Priority Queue (without heapq)

class Patient:

    def __init__(self, patient_id, name, severity, arrival_time):
        self.patient_id = patient_id
        self.name = name
        self.severity = severity  # 1=Critical, 2=High, 3=Medium, 4=Low
        self.arrival_time = arrival_time
        self.status = "Waiting"

    def __repr__(self):
        return f"Patient({self.patient_id}, {self.name}, Severity: {self.severity}, Time: {self.arrival_time})"


class MedicalEmergencyQueue:

    def __init__(self):
        self.queue = []
        self.treated = []

    def add_patient(self, patient_id, name, severity, arrival_time):
        patient = Patient(patient_id, name, severity, arrival_time)
        self.queue.append(patient)
        # Sort queue by severity (lower number = higher priority)
        self.queue.sort(key=lambda p: p.severity)
        severity_map = {1: "🔴 CRITICAL", 2: "🟠 HIGH", 3: "🟡 MEDIUM", 4: "🟢 LOW"}
        print(f"✅ Patient Added: {name} (ID: {patient_id}) - {severity_map[severity]}")

    def treat_next_patient(self):
        if not self.is_empty():
            patient = self.queue.pop(0)
            patient.status = "Treated"
            self.treated.append(patient)
            severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
            print(f"🏥 Now Treating: {patient.name} (Severity: {severity_map[patient.severity]})")
            return patient
        else:
```

```python
class MedicalEmergencyQueue:
            return None

    def peek_next_patient(self):
        if not self.is_empty():
            patient = self.queue[0]
            severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
            print(f"1️⃣ Next Patient: {patient.name} (Severity: {severity_map[patient.severity]})")
            return patient
        else:
            print("❌ No patients in queue.")
            return None

    def is_empty(self):
        return len(self.queue) == 0

    def display_queue(self):
        if self.is_empty():
            print("📭 No patients waiting.\n")
        else:
            print("\n🚑 EMERGENCY QUEUE (Sorted by Priority):")
            print("-" * 80)
            severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
            for patient in self.queue:
                print(f"ID: {patient.patient_id:<6} | Name: {patient.name:<20} | Severity: {severity_map[patient.severity]:<10} | Time: {patient.arrival_time}")
            print("-" * 80 + "\n")

    def display_treated_patients(self):
        if not self.treated:
            print("📭 No treated patients yet.\n")
        else:
            print("\n✅ TREATED PATIENTS:")
            print("-" * 80)
            severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
            for patient in self.treated:
                print(f"ID: {patient.patient_id:<6} | Name: {patient.name:<20} | Severity: {severity_map[patient.severity]}")
            print("-" * 80 + "\n")
```

```python
# Example Usage
if __name__ == "__main__":
    print("=" * 80)
    print("🏥 SMART UNIVERSITY HOSTEL MEDICAL EMERGENCY SYSTEM")
    print("=" * 80 + "\n")

    hospital = MedicalEmergencyQueue()

    # Add patients with different severity levels
    print("📝 Adding Patients to Emergency Queue:\n")
    hospital.add_patient(1, "John Doe", 4, "09:00")
    hospital.add_patient(2, "Sarah Smith", 1, "09:05")
    hospital.add_patient(3, "Mike Johnson", 3, "09:10")
    hospital.add_patient(4, "Emily Brown", 2, "09:15")
    hospital.add_patient(5, "David Wilson", 1, "09:20")

    # Display queue
    hospital.display_queue()

    # Peek next patient
    print("🔍 Checking Next Patient:\n")
    hospital.peek_next_patient()

    # Treat patients
    print("\n🏥 TREATMENT PROCESS:\n")
    hospital.treat_next_patient()
    hospital.treat_next_patient()
    hospital.treat_next_patient()

    # Display remaining queue
    hospital.display_queue()

    # Display treated patients
    hospital.display_treated_patients()

    # Treat remaining patients
    print("Treating remaining patients:\n")
    hospital.treat_next_patient()
    hospital.treat_next_patient()

    hospital.display_treated_patients()
```

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
================================================================================
🏥 SMART UNIVERSITY HOSTEL MEDICAL EMERGENCY SYSTEM
================================================================================

📋 Adding Patients to Emergency Queue:

✅ Patient Added: John Doe (ID: 1) - 🟢 LOW
✅ Patient Added: Sarah Smith (ID: 2) - 🔴 CRITICAL
✅ Patient Added: Mike Johnson (ID: 3) - 🟡 MEDIUM
✅ Patient Added: Emily Brown (ID: 4) - 🟠 HIGH
✅ Patient Added: David Wilson (ID: 5) - 🔴 CRITICAL

🚑 EMERGENCY QUEUE (Sorted by Priority):
--------------------------------------------------------------------------------
ID: 2      | Name: Sarah Smith          | Severity: CRITICAL    | Time: 09:05
ID: 5      | Name: David Wilson         | Severity: CRITICAL    | Time: 09:20
ID: 4      | Name: Emily Brown          | Severity: HIGH        | Time: 09:15
ID: 3      | Name: Mike Johnson         | Severity: MEDIUM      | Time: 09:10
ID: 1      | Name: John Doe             | Severity: LOW         | Time: 09:00
--------------------------------------------------------------------------------

🔍 Checking Next Patient:

👀 Next Patient: Sarah Smith (Severity: CRITICAL)

🏥 TREATMENT PROCESS:

🏥 Now Treating: Sarah Smith (Severity: CRITICAL)
🏥 Now Treating: David Wilson (Severity: CRITICAL)
🏥 Now Treating: Emily Brown (Severity: HIGH)

🚑 EMERGENCY QUEUE (Sorted by Priority):
--------------------------------------------------------------------------------
ID: 3      | Name: Mike Johnson         | Severity: MEDIUM      | Time: 09:10
ID: 1      | Name: John Doe             | Severity: LOW         | Time: 09:00
--------------------------------------------------------------------------------
```

```
✅ TREATED PATIENTS:
--------------------------------------------------------------------------------
ID: 2      | Name: Sarah Smith          | Severity: CRITICAL
ID: 5      | Name: David Wilson         | Severity: CRITICAL
ID: 4      | Name: Emily Brown          | Severity: HIGH
--------------------------------------------------------------------------------

Treating remaining patients:

🏥 Now Treating: Mike Johnson (Severity: MEDIUM)
🏥 Now Treating: John Doe (Severity: LOW)

✅ TREATED PATIENTS:
--------------------------------------------------------------------------------
ID: 2      | Name: Sarah Smith          | Severity: CRITICAL
ID: 5      | Name: David Wilson         | Severity: CRITICAL
ID: 4      | Name: Emily Brown          | Severity: HIGH
ID: 3      | Name: Mike Johnson         | Severity: MEDIUM
ID: 1      | Name: John Doe             | Severity: LOW
--------------------------------------------------------------------------------

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```

<u>Task #19: Smart Electricity Distribution System</u>

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling

2. Fault Priority Handling

3. Consumer Record Lookup

4. Power Grid Connectivity

5. Meter Reading History Tracking

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
"""
FEATURE MAPPING TABLE:
====================
1. Complaint Handling → Queue
    Justification: Complaints must be addressed in FIFO order based on arrival time.
    Queue ensures fair service and maintains chronological order without bias.

2. Fault Priority Handling → Priority Queue
    Justification: Critical faults need expedited repair regardless of arrival time.
    Priority queue sorts by severity level to address urgent faults first.

3. Consumer Record Lookup → Hash Table
    Justification: Provides O(1) average lookup time using consumer ID as key.
    Enables instant retrieval of consumer details, billing info, and account status.

4. Power Grid Connectivity → Graph
    Justification: Substations are nodes and transmission lines are edges in the network.
    Graph structure enables pathfinding and connectivity analysis for grid management.

5. Meter Reading History Tracking → Linked List
    Justification: Maintains chronological order of meter readings with easy insertion.
    Linked list efficiently stores historical data without requiring contiguous memory.
"""

# IMPLEMENTATION: Complaint Handling System using Queue

class Complaint:

    def __init__(self, complaint_id, consumer_id, consumer_name, issue, reported_time):
        self.complaint_id = complaint_id
        self.consumer_id = consumer_id
        self.consumer_name = consumer_name
        self.issue = issue
        self.reported_time = reported_time
        self.status = "Pending"

    def __repr__(self):
        return f"Complaint({self.complaint_id}, {self.consumer_name}, {self.issue})"


class ComplaintHandlingQueue:

    def __init__(self):
        self.queue = []
        self.resolved = []

    def register_complaint(self, complaint_id, consumer_id, consumer_name, issue, reported_time):
        complaint = Complaint(complaint_id, consumer_id, consumer_name, issue, reported_time)
        self.queue.append(complaint)
        print(f"✅ Complaint registered: ID {complaint_id} - {consumer_name} ({issue})")

    def resolve_complaint(self):
        if not self.is_empty():
            complaint = self.queue.pop(0)
            complaint.status = "Resolved"
            self.resolved.append(complaint)
            print(f"✔  Resolved: Complaint {complaint.complaint_id} - {complaint.consumer_name}")
            return complaint
        else:
            print("⚠  No complaints in queue.")
            return None
```

```python
        return None

    def peek_next_complaint(self):
        if not self.is_empty():
            complaint = self.queue[0]
            print(f"📋 Next complaint: ID {complaint.complaint_id} - {complaint.consumer_name} ({complaint.issue})")
            return complaint
        else:
            print("⚠️  Queue is empty.")
            return None

    def is_empty(self):
        return len(self.queue) == 0

    def get_queue_length(self):
        return len(self.queue)

    def display_pending_complaints(self):
        if self.is_empty():
            print("📭 No pending complaints.\n")
        else:
            print("\n⚡ PENDING COMPLAINTS QUEUE (FIFO):")
            print("-" * 100)
            for i, complaint in enumerate(self.queue, 1):
                print(f"{i}. ID: {complaint.complaint_id:<8} | Consumer: {complaint.consumer_name:<20} | Issue: {complaint.issue:<30} | Time: {complaint.reported_time}")
            print("-" * 100 + "\n")

    def display_resolved_complaints(self):
        if not self.resolved:
            print("📭 No resolved complaints yet.\n")
        else:
            print("\n✅ RESOLVED COMPLAINTS:")
            print("-" * 100)
            for i, complaint in enumerate(self.resolved, 1):
                print(f"{i}. ID: {complaint.complaint_id:<8} | Consumer: {complaint.consumer_name:<20} | Issue: {complaint.issue:<30}")
            print("-" * 100 + "\n")
```

```python
# Example Usage
if __name__ == "__main__":
    print("=" * 100)
    print("⚡ SMART ELECTRICITY DISTRIBUTION SYSTEM - COMPLAINT HANDLING QUEUE")
    print("=" * 100 + "\n")

    system = ComplaintHandlingQueue()

    # Register complaints
    print("📝 Registering complaints:\n")
    system.register_complaint(1001, 5001, "Raj Kumar", "Power outage", "09:00 AM")
    system.register_complaint(1002, 5002, "Priya Singh", "High billing", "09:15 AM")
    system.register_complaint(1003, 5003, "Amit Patel", "Meter malfunction", "09:30 AM")
    system.register_complaint(1004, 5004, "Sneha Gupta", "Damaged pole", "09:45 AM")
    system.register_complaint(1005, 5005, "Vikram Shah", "Connection issue", "10:00 AM")

    # Display pending complaints
    system.display_pending_complaints()

    # Peek at next complaint
    print("🔍 Checking next complaint:\n")
    system.peek_next_complaint()
    print(f"\n📊 Total Pending Complaints: {system.get_queue_length()}\n")

    # Resolve complaints
    print("🔧 RESOLVING COMPLAINTS:\n")
    system.resolve_complaint()
    system.resolve_complaint()
    system.resolve_complaint()

    # Display updated queue
    system.display_pending_complaints()

    # Display resolved complaints
    system.display_resolved_complaints()

    # Resolve remaining complaints
    print("Resolving remaining complaints:\n")
    system.resolve_complaint()
    system.resolve_complaint()

    system.display_pending_complaints()
    system.display_resolved_complaints()

    print("✅ All complaints have been resolved!")
```

Output:

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desk
top/AIAC/assg_11.py
==========================================================================================
⚡ SMART ELECTRICITY DISTRIBUTION SYSTEM - COMPLAINT HANDLING QUEUE
==========================================================================================

📋 Registering complaints:

✅ Complaint registered: ID 1001 - Raj Kumar (Power outage)
✅ Complaint registered: ID 1002 - Priya Singh (High billing)
✅ Complaint registered: ID 1003 - Amit Patel (Meter malfunction)
✅ Complaint registered: ID 1004 - Sneha Gupta (Damaged pole)
✅ Complaint registered: ID 1005 - Vikram Shah (Connection issue)

⚡ PENDING COMPLAINTS QUEUE (FIFO):
------------------------------------------------------------------------------------------
1. ID: 1001     | Consumer: Raj Kumar           | Issue: Power outage                  | Time: 09:00 AM
2. ID: 1002     | Consumer: Priya Singh         | Issue: High billing                  | Time: 09:15 AM
3. ID: 1003     | Consumer: Amit Patel          | Issue: Meter malfunction             | Time: 09:30 AM
4. ID: 1004     | Consumer: Sneha Gupta         | Issue: Damaged pole                  | Time: 09:45 AM
5. ID: 1005     | Consumer: Vikram Shah         | Issue: Connection issue              | Time: 10:00 AM
------------------------------------------------------------------------------------------

🔍 Checking next complaint:

👀 Next complaint: ID 1001 - Raj Kumar (Power outage)

📊 Total Pending Complaints: 5

🔧 RESOLVING COMPLAINTS:

✔ Resolved: Complaint 1001 - Raj Kumar
✔ Resolved: Complaint 1002 - Priya Singh
✔ Resolved: Complaint 1003 - Amit Patel

⚡ PENDING COMPLAINTS QUEUE (FIFO):
------------------------------------------------------------------------------------------
1. ID: 1004     | Consumer: Sneha Gupta         | Issue: Damaged pole                  | Time: 09:45 AM
2. ID: 1005     | Consumer: Vikram Shah         | Issue: Connection issue              | Time: 10:00 AM
------------------------------------------------------------------------------------------
```

```
✅ RESOLVED COMPLAINTS:
------------------------------------------------------------------------------------------
1. ID: 1001     | Consumer: Raj Kumar           | Issue: Power outage
2. ID: 1002     | Consumer: Priya Singh         | Issue: High billing
3. ID: 1003     | Consumer: Amit Patel          | Issue: Meter malfunction
------------------------------------------------------------------------------------------

Resolving remaining complaints:

✔ Resolved: Complaint 1004 - Sneha Gupta
✔ Resolved: Complaint 1005 - Vikram Shah
📭 No pending complaints.


✅ RESOLVED COMPLAINTS:
------------------------------------------------------------------------------------------
1. ID: 1001     | Consumer: Raj Kumar           | Issue: Power outage
2. ID: 1002     | Consumer: Priya Singh         | Issue: High billing
3. ID: 1003     | Consumer: Amit Patel          | Issue: Meter malfunction
4. ID: 1004     | Consumer: Sneha Gupta         | Issue: Damaged pole
5. ID: 1005     | Consumer: Vikram Shah         | Issue: Connection issue
------------------------------------------------------------------------------------------

✅ All complaints have been resolved!
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```